

1. Definicja i własności protokołu kryptograficznego.

DEFINICJA PROTOKOŁU

Protokół jest to szereg kroków, obejmujących dwie lub więcej stron, podejmowanych w celu realizacji zadania.

Inaczej mówiąc jest to sekwencja działań z których każde musi być kolejno wykonane i żadne nie może być podjęte, zanim poprzednie nie zostanie ukończone.

Protokół kryptograficzny jest to protokół wykorzystujący kryptografię.

WŁAŚCIWOŚCI PROTOKOŁU

1. Każdy użytkownik (uczestnik) musi go znać i kolejno wykonywać wszystkie kroki.
2. Każdy użytkownik musi zgodzić się na jego stosowanie.
3. Protokół nie może być mylący, każdy krok powinien być dobrze zdefiniowany i nie może wystąpić jakakolwiek szansa na nieporozumienie.
4. Protokół musi być kompletny, dla każdej możliwej sytuacji musi być podany odpowiedni sposób postępowania.

ZAGROŻENIA DLA BEZPIECZEŃSTWA INFORMACJI OBEJMUJĄ ZAGROŻENIA DLA

- ▶ poufności,
- ▶ integralności,
- ▶ dostępności,
- ▶ autentyczności,
- ▶ niezaprzeczalności,
- ▶ rozliczalności,
- ▶ niezawodności.

2. Typy protokołów kryptograficznych (z przykładami).

1. PROTOKOŁY ARBITRAŻOWE

- A) Wymagają istnienia osoby trzeciej będącej stroną w protokole, jednak nie związanej z żadną ze stron, obdarzonej zaufaniem.
- B) strony protokołu przyjmują bez zastrzeżeń wszystkie orzeczenia dotyczące:
 - ▶ prawdziwości wypowiedzi,
 - ▶ poprawności uczynków,
 - ▶ poprawności zakończenia przewidzianej dla danej strony części protokołu.

PRZYKŁAD 1.

Cel: Bob chce kupić samochód od Alicji za 10000. Alicja chce być pewna, że Bob jest wypłacalny (czek Boba ma pokrycie).
Zaufaną trzecią stroną jest bank.

KROKI PROTOKOŁU:

1. Bob wypisuje czek na sumę 10000 i znosi go do banku.
2. Bank potwierdza czek Boba (jeżeli na koncie Boba znajduje się odpowiednia kwota ≥ 10000). Bank blokuje kwotę 10000 na koncie Boba do momentu realizacji czeku.
3. Bob przekazuje potwierdzony czek Alicji. Alicja przekazuje samochód (wraz z dokumentami) Bobowi.
4. Alicja realizuje czek w banku.

2. PROTOKOŁY ROZJEMCZE

- A) Arbiter biorący udział w protokole rozjemczym nazywany jest sędzią.
- B) Sędzia nie jest stałym uczestnikiem protokołu, jest on bezpośrednio zatrudniany tylko w przypadku sporów między stronami.
- C) Sędzia wydaje orzeczenia o poprawności albo nie dokonaniu transakcji.

3. PROTOKOŁY SAMOWYMUSZAJĄCE

- A) Strony porozumiewają się bezpośrednio.
- B) W celu zakończenia działania protokołu nie wymaga się obecności arbitra.
- C) Do rozwiązywania sporów nie jest potrzebny sędzia.
- D) W przypadku oszustwa jednej ze stron, druga strona jest w stanie to wykryć (i przerwać wykonywanie protokołu).

3. Definicja i własności protokołów uzgadniania klucza.

DEFINICJA PROTOKOŁU UZGADNIANIA KLUCZA

Protokoły uzgadniania klucza służą dwóm (albo więcej) stronom chcącym komunikować się poprzez niezabezpieczony kanał do wygenerowania wspólnego tajnego klucza (na potrzeby algorytmu szyfrowania).

Protokoły uzgadniania klucza są podstawą dla algorytmów kryptograficznych wykorzystujących współdzielone klucze. Algorytmy te służą ochronie przesyłanych przez niezabezpieczone sieci danych.

WŁASNOŚCI PROTOKOŁÓW UZGADNIANIA KLUCZA

Podstawowe własności związane z bezpieczeństwem:

- ▶ Domniemane uwierzytelnienie klucza (Implicit key authentication).
- ▶ Bezpośrednie uwierzytelnienie klucza (Explicit key authentication).

Dodatkowe własności związane z bezpieczeństwem

- ▶ Known-key security.
- ▶ Forward secrecy.
- ▶ Key-compromise impersonation resilience.
- ▶ Unknown key-share resilience.

DOMNIEMANE UWIERZYTELNIENIE KLUCZA

Mówimy, że protokół uzgadniania klucza zapewnia domniemane uwierzytelnienie klucza (dla strony **A** w stosunku do strony **B**) jeśli strona **A** jest pewna że jedynie strona **B** może poznać wartość konkretnego tajnego klucza.

BEZPOŚREDNIE UWIERZYTELNIENIE KLUCZA

Mówimy, że protokół uzgadniania klucza zapewnia bezpośrednie potwierdzenie posiadania klucza (dla strony **A** w stosunku do strony **B**) jeżeli strona **A** jest pewna, że strona **B** wyznaczyła wartość uzgadnianego tajnego klucza.

Protokół zapewnia domniemane uwierzytelnienie klucza jeżeli strona **A** jest pewna, że jedynie strona **B** mogła wyznaczyć współdzielony klucz.

Mówimy, że protokół uzgadniania klucza zapewnia bezpośrednie uwierzytelnienie klucza (dla strony **A** w stosunku do strony **B**) jeżeli zapewnia zarówno domniemane uwierzytelnienie klucza jak i bezpośrednie potwierdzenie posiadania klucza (dla strony **A** w stosunku do strony **B**).

KNOWN-KEY SECURITY

Każde wykonanie protokołu uzgadniania klucza służy wygenerowaniu unikalnego tajnego klucza (klucza sesji).

Klucze sesji są wymagane w celu ograniczenia ilości szyfrogramów, które można wykorzystać w atakach kryptograficznych oraz ograniczeniu informacji, jakie można odzyskać w przypadku znalezienia klucza (przez atakującego).

Protokół powinien spełniać swoje zadanie nawet jeżeli przeciwnik poznał pewną liczbę wcześniej wygenerowanych kluczy sesji.

FORWARD SECRECY

Ujawnienie długoterminowego klucza prywatnego jednego z uczestników protokołu nie powinno wpływać na ujawnienie wcześniej wygenerowanych kluczy sesyjnych.

Rozróżniane są tutaj dwa scenariusze:

- ▶ ujawniony został prywatny klucz długoterminowy jednej ze stron – half forward secrecy;
- ▶ długoterminowe klucze obu stron protokołu zostały ujawnione – full forward secrecy.

KEY-COMPROMISE IMPERSONATION RESILIENCE

Założmy, że atakujący poznał długoterminowy klucz prywatny strony **A**. Może on od tej pory udawać stronę **A**.

Własność ta oznacza, że atakujący nie może udawać innych użytkowników w stosunku do strony **A**.

KEY-COMPROMISE IMPERSONATION RESILIENCE

Założmy, że atakujący poznał długoterminowy klucz prywatny strony **A**. Może on od tej pory udawać stronę **A**.

Własność ta oznacza, że atakujący nie może udawać innych użytkowników w stosunku do strony **A**.

4. Protokół Diffiego-Hellmana i atak „man in the middle”.

WPROWADZENIE

Protokół Diffiego-Hellmana pozwala stronom **A** i **B** na wygenerowanie współdzielonego klucza sesji **K** wykorzystując do wymiany danych publiczny (niezabezpieczony) kanał łączności.

Klucz **K** powinien pozostać nieznany dla dowolnego podsłuchującego wymianę informacji pomiędzy stronami protokołu.

PARAMETRY DZIEDZINY

- ▷ Bezpieczeństwo klucza generowanego w trakcie realizacji protokołu bazuje na problemie logarytmu dyskretnego.
- ▷ Przed przystąpieniem do realizacji protokołu strony A i B muszą uzgodnić grupę cykliczną $G_n = \langle g \rangle$ generowaną przez element g . Rząd grupy oznaczony jest przez n (najczęściej wymagane jest, aby n było dużą liczbą pierwszą).

UWAGA

Wyznaczenie logarytmu dyskretnego w tej grupie powinno być obliczeniowo niemożliwe tj. dla dowolnego $a \in G_n$ powinno być obliczeniowo niemożliwe wyznaczenie takiej liczby całkowitej α spełniającej:

$$a = g^\alpha. \quad (1)$$

KROKI PROTOKOŁU:

1. Strona A generuje losową liczbę całkowitą $x_A \in \mathbb{Z}_n^*$, oblicza

$$K_A = g^{x_A} \quad (2)$$

i wysyła K_A do strony B.

2. Strona B generuje losową liczbę całkowitą $x_B \in \mathbb{Z}_n^*$, oblicza

$$K_B = g^{x_B} \quad (3)$$

i wysyła K_B do strony A.

KROKI PROTOKOŁU:

3. A oblicza klucz sesyjny:

$$K = (K_B)^{x_A} = (g^{x_B})^{x_A} = g^{x_A x_B}. \quad (4)$$

4. B oblicza klucz sesyjny:

$$K = (K_A)^{x_B} = (g^{x_A})^{x_B} = g^{x_A x_B}. \quad (5)$$

MAN IN THE MIDDLE

ZAŁOŻENIA

- ▷ Atakujący M ma możliwość czytania, modyfikowania i usuwania wiadomości przesyłanych pomiędzy stronami A i B / Ma także możliwość generowania nowych wiadomości.
- ▷ W tym ataku strona M w komunikacji ze stroną A udaje stronę B . W komunikacji ze stroną B atakujący M udaje stronę A .

KROKI ATAKU

1. Strona A generuje losową liczbę całkowitą $x_A \in \mathbb{Z}_n^*$, oblicza

$$K_A = g^{x_A} \quad (6)$$

i wysyła K_A do strony B .

2. M przechwytuje K_A , generuje losową liczbę całkowitą $x_M \in \mathbb{Z}_n^*$, oblicza

$$K_M = g^{x_M} \quad (7)$$

i wysyła K_M do strony B .

3. Strona B generuje losową liczbę całkowitą $x_B \in \mathbb{Z}_n^*$, oblicza

$$K_B = g^{x_B} \quad (8)$$

i wysyła K_B do strony A .

4. M przechwytuje K_B i wysyła K_M do strony A .

5. A oblicza klucz sesyjny:

$$K' = (K_M)^{x_A} = (g^{x_M})^{x_A} = g^{x_A x_M}. \quad (9)$$

6. B oblicza klucz sesyjny:

$$K'' = (K_M)^{x_B} = (g^{x_M})^{x_B} = g^{x_B x_M}. \quad (10)$$

7. M oblicza klucze:

$$K' = (K_A)^{x_M} = g^{x_A x_M}, \quad K'' = (K_B)^{x_M} = g^{x_B x_M} \quad (11)$$

do komunikacji z A (K') i z B (K'').

PARAMETRY DZIEDZINY

- ▷ Bezpieczeństwo klucza generowanego w trakcie realizacji protokołu bazuje na problemie logarytmu dyskretnego.
- ▷ Przed przystąpieniem do realizacji protokołu strony *A* i *B* muszą uzgodnić grupę cykliczną $G_n = \langle g \rangle$ generowaną przez element g . Rząd grupy oznaczony jest przez n (najczęściej wymagane jest, aby n było dużą liczbą pierwszą).
- ▷ Każda strona posiada klucz prywatny na potrzeby algorytmu podpisu *SIGN*. Wszyscy użytkownicy posiadają klucze publiczne innych potrzebne w algorytmie weryfikacji podpisu *VER*.
- ▷ Przed rozpoczęciem realizacji protokołu obie strony ustalają identyfikator sesji s .

KROKI PROTOKOŁU:

1. Strona *A* generuje losową liczbę całkowitą $x_A \in \mathbb{Z}_n^*$, oblicza

$$K_A = g^{x_A} \quad (12)$$

i wysyła (A, s, K_A) do strony *B*.

2. Strona *B* generuje losową liczbę całkowitą $x_B \in \mathbb{Z}_n^*$, oblicza

$$K_B = g^{x_B}, \quad \text{sign}_B = \text{SIGN}_{B_{\text{priv}}}(B, s, K_B, K_A, A) \quad (13)$$

i wysyła (B, s, K_B) oraz sign_B do strony *A*. Ponadto *B* oblicza klucz sesji:

$$K = (K_A)^{x_B} = g^{x_A x_B}. \quad (14)$$

3. *A* weryfikuje podpis sign_B wykorzystując klucz publiczny B_{pub} strony *B*. Jeżeli weryfikacja kończy się wynikiem pozytywnym *A* oblicza:

$$K = (K_B)^{x_A} = g^{x_A x_B}, \quad \text{sign}_A = \text{SIGN}_{A_{\text{priv}}}(A, s, K_A, K_B, B). \quad (15)$$

i wysyła (A, s, sign_A) do *B*.

4. *B* weryfikuje podpis sign_A wykorzystując klucz publiczny A_{pub} strony *A*. Jeżeli weryfikacja kończy się wynikiem pozytywnym, to K jest kluczem sesyjnym dla sesji s .

5. Definicja uwierzytelnienia, na czym może być ono oparte, przykładowe protokoły uwierzytelniania.

WPROWADZENIE

Identyfikacja użytkowników i systemu wobec użytkownika jest obecnie niezbędna w zaawansowanych rozwiązaniach systemów kryptograficznych.

Z jednej strony dostęp do systemu powinni mieć tylko uprawnieni użytkownicy, natomiast z drugiej strony użytkownik powinien mieć pewność, że komunikuje się z wykorzystaniem systemu upoważnionego do tego celu.

Zdarzały się przypadki np. podkładania fałszywych bankomatów zbierających dane o kartach użytkowników i ich PIN-ach.

ROZRÓŻNIAMY IDENTYFIKACJĘ UŻYTKOWNIKA:

- ▶ w kryptografii z kluczem publicznym,
- ▶ w kryptografii z kluczem symetrycznym.

OGÓLNE UWIERZYTELNIENIE MOŻE BYĆ OPARTE O:

- ▶ **Wiedzę:** np. hasła;
- ▶ Posiadanie: np. fizyczne klucze lub karty;
- ▶ Cechy niezmiennialne: np. biometryczne takie jak odcisk palca itp.
- ▶ Lokalizację: uwierzytelnienie oparte jest o fakt, że użytkownik znajduje się w specyficznym miejscu (np. bardzo rzadko sprawdzana jest tożsamość urzędnika jeżeli znajduje się on w urzędzie).
- ▶ Delegację uwierzytelnienia: osoba sprawdzająca uwierzytelnia użytkownika na podstawie uwierzytelnienia przeprowadzonego wcześniej przez inną zaufaną stronę.

W przypadku sieci telekomunikacyjnych bezpośrednie uwierzytelnienie oparte o powyższe metody może być trudne do przeprowadzenia albo nie zapewnia bezpieczeństwa.

Rozwiązanie: protokoły kryptograficzne.

GŁÓWNE METODY UWIERZYTELNIENIA:

- ▶ Uwierzytelnienie arbitrażowe: arbiter (zaufana trzecia strona) jest bezpośrednio zaangażowany w wykonanie protokołu uwierzytelniania;
- ▶ Uwierzytelnienie bezpośrednie: strony **A** i **B** bezpośrednio wzajemnie się uwierzytelniają.

UWIERZYTELNIENIE ARBITRAŻOWE

Zalety:

- ▶ Pozwala stronom **A** i **B** na wzajemne uwierzytelnienie nawet jeżeli nie ustaliły one wcześniej wspólnego sekretu;
- ▶ Pozwala nie znającym się wcześniej stronom na wykorzystanie kryptografii symetrycznej.

Wady:

- ▶ Arbiter może być wąskim gardłem, dostępność arbitra jest krytyczna dla wykonania protokołu uwierzytelniania;
- ▶ Arbiter może monitorować całą aktywność związaną z uwierzytelnianiem.

UWIERZYTELNIENIE BEZPOŚREDNIE

Zalety:

- ▶ Brak konieczności udziału arbitra (brak potencjalnego efektu wąskiego gardła generowanego przez arbitra);

Wady:

- ▶ Wymaga użycia kryptografii klucza publicznego albo wcześniej ustalonego wspólnego sekretu.

PODSTAWOWY PROTOKÓŁ UWIERZYTELNIENIE UŻYTKOWNIKA:

1. użytkownik wprowadza swoje hasło do komputera,
2. komputer oblicza pewną funkcję jednokierunkową hasła,
3. komputer porównuje wynik funkcji z wartością przechowywaną w pamięci,
4. w przypadku zgodności umożliwia pracę użytkownikowi w ramach jego uprawnień.

PRZYKŁAD PROTOKOŁ LAMPORTA

ZAŁOŻENIA

- ▶ Wykorzystuje on funkcję jednokierunkową do identyfikacji użytkownika. Użytkownik musi zapamiętać i zachować w tajemnicy pewną wartość początkową x_0 .
- ▶ Kolejne wartości x_k w kolejnych krokach będą powstawać jako wyniki działania funkcji jednokierunkowej z argumentem x_{k-1} .
- ▶ Użytkownik na początku musi wygenerować n iteracji (n – z ustalonego wcześniej zakresu) funkcji f .
- ▶ Następnie podaje wartość x_n systemowi.
- ▶ Przy pierwszym logowaniu wartość ta będzie funkcjonować jako identyfikator użytkownika, natomiast x_{n-1} jako hasło.

OPIS PROTOKOŁU:

1. użytkownik przesyła aktualny identyfikator x_k ,
2. system sprawdza istnienie użytkownika o otrzymanym identyfikatorze, po czym żąda podania hasła czyli x_{k-1} ,
3. użytkownik podaje hasło x_{k-1} ,
4. system weryfikuje poprawność hasła, sprawdzając, czy $f(x_{k-1}) = x_k$, jeśli tak, to zapamiętuje x_{k-1} jako identyfikator przy następnym logowaniu.

PRZYKŁADOWE PROTOKOŁY:

- 1) Z współdzielonym kluczem
- 2) Identyfikacja z kluczem tajnym
- 3) SKID2/3
- 4) Kerberos
- 5) X.509
- 6) SSL

WZAJEMNE UWIERZYTELNIENIE Z WYKORZYSTANIEM PROTOKOŁU BLOKUJĄCEGO

OPIS PROTOKOŁU:

1. użytkownicy A i B wymieniają klucze publiczne,
2. użytkownik A szyfruje wartość P_A (np. swoje hasło) przy użyciu klucza publicznego użytkownika B i wysyła do B połowę wyniku,
3. użytkownik B szyfruje swoją wartość P_B (np. swoje hasło) przy użyciu klucza publicznego A i wysyła do A połowę wyniku,
4. użytkownik A po otrzymaniu pierwszej połowy od B wysyła do B drugą połowę zaszyfrowanej wartości P_A ,
5. użytkownik B otrzymuje całość szyfrogramu i odszyfrowuje wartość P_A , sprawdzając czy jest ona poprawna, po uzyskaniu pozytywnego wyniku wysyła do A swoją drugą połowę zaszyfrowanej wartości P_B ,
6. A otrzymuje całość szyfrogramu od B, deszyfrowuje wartość P_B , sprawdzając jej poprawność.

6. Definicja podziału wiadomości, przykład protokołu.

DEFINICJA PODZIAŁU WIADOMOŚCI.

Podział wiadomości (ang. Secret splitting) ma na celu takie ukrycie informacji pomiędzy n użytkownikami, aby odtworzenie danej wiadomości wymagało współpracy wszystkich uczestników (m.in. ich „części” wiadomości).

PROSTY PRZYKŁAD

OPIS PROTOKOŁU:

1. użytkownik T chce dokonać podziału wiadomości M między użytkowników A i B , w tym celu generuje ciąg losowy R o tej samej długości co M ,
2. użytkownik T oblicza sumę modulo 2 ciągów M i R , tworząc P ,
3. wysyła P użytkownikowi A , natomiast R użytkownikowi B (może również dokonać tego na odwrót),
4. użytkownicy A i B by odtworzyć wiadomość muszą wykonać sumę modulo 2 ciągu P i R .

Inaczej to interpretując jest to system z kluczem jednokrotnym.

DLA n UŻYTKOWNIKÓW

OPIS PROTOKOŁU:

Podobny system można wykorzystać do podziału wiadomości pomiędzy n użytkowników.

1. Należy wtedy wygenerować $n - 1$ ciągów losowych o długości równej długości wiadomości. Wtedy P jest równe sumie modulo 2 wiadomości i wszystkich ciągów losowych.
2. Następnie należy rozesłać P i pozostałe $n - 1$ ciągów losowych pomiędzy użytkownikami.

WŁASNOŚCI

Protokół ten zalicza się do protokołów rozjemczych i użytkownik T panuje nad wszystkim. W protokole tym T musi być uczciwy i cieszyć się zaufaniem. Jest to również protokół z rodzaju wszyscy albo nikt.

7. Definicja schematów progowych, przykład.

DEFINICJA

Innym zagadnieniem są podziały progowe, gdzie wiadomość dzielimy na „części” które rozdzielamy pomiędzy n użytkowników i ustalamy, że jeżeli zbierze się k lub więcej dowolnych użytkowników ($k \leq n$), to mogą oni odtworzyć wiadomość.

„Części” otrzymane z podziału wiadomości nazywa się cieniami.

UWAGA.

Tego typu schematy nazywa się schematami (k, n) -progowymi.

PRZYKŁAD Z LAGRANGEM

OPIS PROTOKOŁU

Wykorzystuje się tu równania wielomianowe w ciele skończonym. Mamy wiadomość M , którą chcemy podzielić, w tym celu wykonujemy czynności:

1. Wybieramy liczbę pierwszą p , która jest większa niż liczba możliwych cieni i większa niż wartość tajemnicy, która ma być współdzielona.
2. Generowany jest dowolny wielomian stopnia $k - 1$ (dla k z n osób uprawnionych do odtworzenia wiadomości) postaci:

$$f(x) = a_{k-1}x^{k-1} + a_{k-2}x^{k-2} + \dots + a_1x + M \pmod{p} \quad (1)$$

gdzie współczynniki $a_{k-1}, a_{k-2}, \dots, a_1 \in \mathbb{Z}_p$ są wybrane losowo i utrzymane w tajemnicy. Są zapominane po wydaniu cieni osobom upoważnionym, natomiast p jest ujawniane.

3. Wyznaczamy cienie poprzez wyznaczenie wartości wielomianu w n różnych punktach $x_i \in \mathbb{Z}$, $x_i > 0$:

$$l_i = f(x_i) \quad (2)$$

4. Wysyłamy to poszczególnych użytkowników systemu x_i i odpowiadającą jej wartość l_i .
5. Do wyznaczenia M musi się zebrać dowolny podzbiór k osób posiadających cienie. Mogą one wtedy wyznaczyć $a_{k-1}, a_{k-2}, \dots, a_1$ oraz M poprzez odtworzenie postaci wielomianu $f(x)$ (rozwiązanie układu k kongruencji z k niewiadomymi $a_{k-1}, a_{k-2}, \dots, a_1$ i M).

PRZYKŁADOWE SCHEMATY:

- 1) Wektorowy
- 2) Asmutha-Blooma
- 3) Karnina-Green`a-Hellmana

8. Definicja protokołu podpisu niezaprzeczalnego, przykład.

DEFINICJA

Składa się je pod dokumentem w podobnym celu jak zwykły podpis, jednak tym się różni od niego że sprawdzający poprawność podpisu musi skontaktować się z jego wytwórcą celem jego sprawdzenia.

Stawiający podpis ma kontrolę nad jego sprawdzaniem i sprawdzającymi.

PROTOKÓŁ

PODSTAWOWA IDEA PODPISU NIEZAPRZECZALNEGO:

1. Użytkownik **A** przedstawia użytkownikowi **B** podpis cyfrowy,
2. Użytkownik **B** generuje liczbę losową i przesyła ją do **A**,
3. Użytkownik **A** dokonuje obliczeń, wykorzystując liczbę losową i swój klucz prywatny, a potem przesyła wynik do użytkownika **B**, użytkownik **A** mógł wykonać te obliczenia jedynie wtedy, kiedy podpis był poprawny,
4. Użytkownik **B** potwierdza powyższy wynik.

Potwierdzanie dokonywane jest każdorazowo dla każdego użytkownika indywidualnie (musi zaakceptowany przez użytkownika **A**).

SCHEMAT

ZAŁOŻENIA:

Ustalana jest duża liczba pierwsza p i element pierwotny g , są one ujawniane i stosowane przez grupę podpisujących. Użytkownik **A** posiada klucz prywatny x i klucz publiczny $g^x \pmod{p}$.

OPIS ALGORYTMU:

Celem podpisania wiadomości, użytkownik **A** oblicza

$$z = m^x \pmod{p}. \quad (1)$$

METODA KONTROLI PODPISU:

1. Użytkownik B wybiera dwie liczby losowe a i b , obie mniejsze od p , przesyła do użytkownika A wynik działania:

$$c = z^a(g^x)^b \pmod{p}, \quad (2)$$

2. Użytkownik A oblicza $x^{-1} \pmod{p-1}$ i przesyła do użytkownika B wynik działania:

$$d = c^{x^{-1}} \pmod{p} \quad (3)$$

3. Użytkownik B sprawdza, czy:

$$d = m^a g^b \pmod{p}. \quad (4)$$

Poprawny wynik sprawdzenia oznacza zaakceptowanie podpisu.

WŁASNOŚCI

Użytkownik B nie może na podstawie otrzymanych danych z powyższych kroków przekonywać postronne osoby o poprawności podpisu użytkownika A.

Każdy z pozostałych użytkowników chcących sprawdzić poprawność postawionego podpisu musi wykonać powyższe kroki protokołu osobiście, natomiast użytkownik A ma kontrolę nad tym, kto taką kontrolę podpisu chce zrealizować.

DEFINICJA

Innym rozwiązaniem jest protokół nie tylko sprawdzający poprawność podpisu, ale również służący zaprzeczeniu złożenia podpisu przez daną osobę pod dokumentem.

ZAŁOŻENIA

Wybrano i opublikowano dużą liczbę pierwszą p i element pierwotny g .

Użytkownik A posiada swój klucz prywatny x i klucz publiczny $g^x \pmod{p}$.

OPIS PROTOKOŁU:

Celem podpisania dokumentu użytkownik A oblicza

$$z = m^x \pmod{p}. \quad (5)$$

METODA KONTROLI PODPISU:

1. użytkownik B wybiera dwie liczby losowe a i b , obie mniejsze niż p i przesyła do użytkownika A wynik obliczeń:

$$c = m^a g^b \pmod{p}, \quad (6)$$

2. użytkownik A wybiera liczbę losową q mniejszą od p , oblicza i przesyła do użytkownika B wyniki działań:

$$s_1 = c g^q \pmod{p}, \quad s_2 = (c g^q)^x \pmod{p}; \quad (7)$$

3. użytkownik B przesyła do A wartości a i b , tak więc użytkownik A może potwierdzić, że użytkownik B nie oszukiwał w kroku (1),

4. użytkownik A przesyła do B wartość q , więc użytkownik B może użyć $m^x \pmod{p}$ i odtworzyć s_1 i s_2 , jeżeli

$$s_1 = c g^q \pmod{p}, \quad s_2 = (g^x)^{b+q} z^a \pmod{p} \quad (8)$$

to podpis jest poprawny.

UWAGA:

Użytkownik A może także zaprzeczyć, że zrobił podpis z pod dokumentem m .

9. Definicja przesyłania niezaprzeczalnego.

DEFINICJA

Realizowane jest gdy jedna ze stron ma do przekazania pewną wiadomość (ciąg bitów) za gratyfikacją, druga strona potrzebuje tej wiadomości, nie chce kupować „kota w worku” i strony nie ufają sobie.

ZAŁOŻENIA:

Użytkownik A przesyła grupę wiadomości użytkownikowi B. Użytkownik B otrzymuje podzbiór tych wiadomości, ale użytkownik A nie wie, które z nich on otrzymał.

Najistotniejsze jest tu przesłanie od użytkownika A do B dwóch wiadomości, przy czym użytkownik B otrzymuje jedną z nich, natomiast użytkownik A nie wie którą.

OPIS PROTOKOŁU:

1. użytkownik **A** wytwarza dwie pary kluczy publiczny/prywatny i przesyła oba klucze publiczne użytkownikowi **B**,
2. użytkownik **B** wybiera klucz w algorytmie symetrycznym, losowo pobiera jeden z kluczy publicznych użytkownika **A** i za jego pomocą szyfruje swój klucz algorytmu symetrycznego, przesyła zaszyfrowany klucz użytkownikowi **A** bez wskazania który z jej kluczy publicznych został wykorzystany do szyfrowania;
3. użytkownik **A** deszyfruje klucz użytkownika **B**, używając obu swoich kluczy prywatnych, w jednym z przypadków używa on poprawnego klucza i skutecznie deszyfruje klucz algorytmu symetrycznego użytkownika **B**, ponieważ nie zna który klucz tego algorytmu jest poprawny (oba ciągi wyjściowe podobne są do ciągów losowych) oba ciągi są dla niej równoprawne;
4. użytkownik **A** szyfruje, wykorzystując algorytm symetryczny, jedną wiadomość przy użyciu pierwszej wersji klucza i drugą za pomocą drugiej wersji klucza, oba wyniki przesyła do użytkownika **B**;
5. użytkownik **B** deszyfruje obie wiadomości przy użyciu poprawnego klucza algorytmu symetrycznego, ale w wyniku otrzymuje tylko jedną z dwóch poprawną wiadomość;
6. po zakończeniu protokołu, gdy są znane oba możliwe wyniki przesłania użytkownik **A** może przesłać do **B** swój klucz prywatny, aby mógł on sprawdzić, czy on nie oszukuje.

10. Definicja i własności bezpiecznych obliczeń z udziałem wielu stron.

DEFINICJA

Celem bezpiecznych obliczeń z udziałem wielu stron jest wyznaczenie wartości wybranej przez uczestników funkcji dla ich prywatnych wartości wejściowych,

POŻĄDANE WŁASNOŚCI:

- ▶ Poufność;
- ▶ Poprawność;
- ▶ Niezależność od wartości wejściowych;
- ▶ Gwarancja dostarczenia wyniku;
- ▶ Uczciwość.

POUFNOŚĆ

Żaden z uczestników nie powinien dowiedzieć się więcej niż jest to określone w protokole.

W szczególności jedyne informacje jaki uczestnik może uzyskać o innych uczestnikach są informacjami zawartymi w wyniku zwróconym mu przez protokół.

Przykładowo w przypadku aukcji jedynie oferta wygrywająca jest ujawniana. Na tej podstawie można jasno wywnioskować, że pozostałe oferty były niższe.

Z drugiej strony powinna być to jedyna informacja związana z przegranymi ofertami.

POPRAWNOŚĆ

Uczestnicy muszą mieć pewność, że otrzymany przez nich wynik końcowy jest poprawny.

Dla przykładu związanego z aukcją, własność ta oznacza, że uczestnik z najwyższą ofertą ma zagwarantowaną wygraną w aukcji i nie mogą tego zmienić ani inni licytujący ani osoba prowadząca aukcję,

NIEZALEŻNOŚĆ OD WARTOŚCI WEJŚCIOWYCH

Nieuczciwi uczestnicy muszą wybierać swoje wartości wejściowe niezależnie od wartości wejściowych uczciwych uczestników.

Własność ta jest krytyczna dla zamkniętych aukcji (pierwszej ceny), gdzie oferty są trzymane w tajemnicy (do momentu upłynięcia czasu składania ofert). Licytujący muszą zatem określać swoje oferty niezależnie od pozostałych uczestników.

GWARANCJA DOSTARCZENIA WYNIKU

Nieuczciwi uczestnicy nie mogą uniemożliwić dostarczenie do uczciwych uczestników przeznaczonych dla nich wartości wyjściowych (wyników).

Atakujący nie może przerwać obliczeń poprzez przeprowadzenie ataku typu DOS.

UCZCIWOŚĆ

Nieuczciwi uczestnicy powinni otrzymać swoje wyniki tylko wtedy, gdy uczciwi uczestnicy otrzymają również swoje.

Własność ta jest krytyczna m.in. w przypadku podpisywania kontraktów. Szczególne w przypadku, gdy nieuczciwy uczestnik otrzyma podpisany kontrakt a uczciwy uczestnik go nie otrzyma.

11. Definicja obliczeń dla danych zaszyfrowanych, przykład.

DEFINICJA [OBLICZENIA DLA DANYCH ZASZYFROWANYCH]

Jest to grupa problemów polegających na tym, że zleca się osobie trzeciej obliczenia, a zarazem nie można ujawnić tej osobie argumentu obliczeń.

Dla zlecających natomiast ważny jest wynik obliczeń na znanym tylko dla nich argumentcie.

Przykładem może być tu wyznaczanie logarytmu dyskretnego pewnej wartości x przez inne osoby, bez ujawnienia wartości x .

WYZNACZANIE LOGARYTMU DYSKRETNEGO

ZAŁOŻENIA:

Ustalona i znana jest wszystkim wartość p oraz g , użytkownik A ma pewną liczbę $x < p$ (wartość tajna).

OPIS PROTOKOŁU:

1. Użytkownik A wybiera liczbę losową r mniejszą niż p ,

2. Użytkownik A wykonuje obliczenia:

$$x' = x g^r \pmod{p} \quad (10)$$

3. Użytkownik A prosi użytkownika B o obliczenie wartości

$$e' = \log_g(x') \pmod{p} \quad (11)$$

4. Użytkownik B oblicza e' i przesyła wynik do użytkownika A ,

5. użytkownik A odtwarza e przez wyliczenie $e = e' - r \pmod{p - 1}$.

1

12. Definicja i własności zobowiązań bitowych.

WPROWADZENIE

Problem w tym przypadku polega na tym, że musimy ustalić pewne wartości, których początkowo nie można ujawnić, natomiast druga strona musi mieć pewność, że tych wartości nie zmienimy w trakcie realizacji zadania wspólnego.

Na koniec wartości te są ujawniane celem konfrontacji.

ZAŁOŻENIA OBIEKTÓW STANOWIĄCYCH ZOBOWIĄZANIA BITOWE:

- ▶ użytkownik A może zobowiązać się co do postaci obiektów bitowych,
- ▶ użytkownik A może otworzyć dowolny obiekt bitowy, co do postaci którego zobowiązał się wcześniej, nie może on jednak ujawnić obiektu bitowego, którego wartość dla użytkownika B wynosiła by jednocześnie zero i jeden,
- ▶ użytkownik B nie może dowiedzieć się niczego o tym, w jaki sposób użytkownik A otwiera dowolny nie ujawniony obiekt bitowy, w stosunku do którego dokonał zobowiązania,
- ▶ obiekty bitowe nie zawierają żadnej innej informacji niż ta, która określa wartość zobowiązania bitowego, same obiekty bitowe, jak i proces zobowiązywania się i otwierania obiektu nie są skorelowane z jakąkolwiek inną informacją, którą użytkownik A pragnąłby utrzymać w tajemnicy przed użytkownikiem B .

ZOBOWIĄZANIE BITOWE Z ZASTOSOWANIEM KRYPTOGRAFII SYMETRYCZNEJ

ZAŁOŻENIA:

Wykorzystany jest tu symetryczny algorytm szyfrowania.

OPIS PROTOKOŁU:

1. użytkownik B generuje ciąg znaków R i wysyła go do użytkownika A ,
2. użytkownik A generuje wiadomość zawierającą przewidywany bit (lub grupę bitów) i ciąg losowy otrzymany od użytkownika B , szyfruje wszystko przy pomocy klucza K i przesyła wynik z powrotem do użytkownika B ,

$$E_K(R, b). \quad (1)$$

OPIS PROTOKOŁU:

Użytkownik B nie zna klucza K , więc nie może odszyfrować wartości b . Gdy nadejdzie pora następuje wykonanie ujawnienia wartości b . W tym celu:

1. użytkownik A przesyła do B klucz K ,
2. użytkownik B deszyfruje wiadomość i sprawdza czy wartość R jest taka jaką wysyłał, jeśli tak odczytuje wartość b .

UWAGA

Moc takiego rozwiązania tkwi w mocy algorytmu szyfrowania.

ZOBOWIĄZANIE BITOWE Z ZASTOSOWANIEM FUNKCJI JEDNOKIERUNKOWYCH

ZAŁOŻENIA:

Stosuje się tu funkcje jednokierunkowe.

OPIS PROTOKOŁU:

1. użytkownik **A** generuje dwa ciągi losowe R_1 i R_2 ,
2. użytkownik **A** tworzy wiadomość składającą się z jego ciągów losowych i bitu (bitów) b , który stanowi zobowiązanie,
3. użytkownik **A** oblicza wartość skrótu wiadomości, a użytkownikowi **B** wysyła wynik i jeden z ciągów losowych,

$$H(R_1, R_2, b), R_1. \rightsquigarrow B \quad (2)$$

Wartości te stanowią zobowiązanie strony **A**, Strona **B** nie może na podstawie skrótu i jednej wartości losowej odtworzyć wartości b .

OPIS PROTOKOŁU:

Dokończeniem protokołu powinny być następujące czynności:

1. użytkownik **A** przesyła do użytkownika **B** pierwotną wiadomość:

$$R_1, R_2, b \quad (3)$$

2. użytkownik **B** oblicza wartość skrótu wiadomości, porównuje ją z wcześniej uzyskaną, oraz porównuje R_1 otrzymane wcześniej i obecnie, jeśli wartości te są zgodne, przyjmuje bity b .

UWAGA

Protokół ten wymaga tylko komunikacji jednokierunkowej od użytkownika **A** do **B**. Użytkownik **B** nie musi odpowiadać.

ZOBOWIĄZANIE BITOWE Z ZASTOSOWANIEM GENERATORÓW CIĄGÓW PSEUDOLOSOWYCH

OPIS PROTOKOŁU:

1. użytkownik **B** generuje ciąg znaków i wysyła go do użytkownika **A**,
2. użytkownik **A** wytwarza losową wartość początkową dla generatora bitów pseudolosowych, następnie dla każdego bitu losowego ciągu użytkownika **B** wysyła on do **A** jedną z dwóch wiadomości:
 - 2.1 wartość wyjściową generatora, jeżeli wartość bitu użytkownika **B** wynosi zero, albo
 - 2.2 sumę modulo 2 wartości wyjściowej generatora i bitu użytkownika **A**, jeżeli wartość bitu użytkownika **B** wynosi jeden;

OPIS PROTOKOŁU:

Gdy zajdzie potrzeba zakończenia i ujawnienia bitów użytkownika A, protokół kontynuowany jest w następujących krokach:

1. użytkownik A przesyła do B wygenerowaną przez siebie losową wartość początkową,
2. użytkownik B wykonuje krok (2) w celu potwierdzenia, że użytkownik A postępował uczciwie.

ZAŁOŻENIA BEZPIECZEŃSTWA SYSTEMU:

- ▶ losowy ciąg użytkownika B musi być wystarczająco długi,
- ▶ generator pseudolosowy jest nieprzewidywalny.

13. Definicja, cechy i przykład protokołów rzutu monetą.

CEL

Protokoły tego typu służą do ustalania (losowania) wartości niezależnej od intencji użytkowników protokołów. Mówimy tu o tzw. „rzucaniu monetą”, czyli pewnej modyfikacji lub uzupełnienia protokołów zobowiązania bitowego.

CECHY TEGO TYPU PROTOKOŁÓW:

- ▶ użytkownik A musi losować jakąś wartość („rzucać monetą”), zanim użytkownik B zacznie odgadywać jej wartość,
- ▶ użytkownik A nie może mieć możliwości dokonania ponownego losowania po usłyszeniu orzeczenia użytkownika B,
- ▶ użytkownik B nie może dowiedzieć się, co wylosował użytkownik A zanim podjął decyzję.

ZAŁOŻENIA:

użytkownicy wcześniej ustalili funkcję jednokierunkową.

OPIS PROTOKOŁU:

1. użytkownik A wybiera losową liczbę x , oblicza $y = f(x)$, gdzie $f(x)$ jest funkcją jednokierunkową i przesyła wartość y do użytkownika B,
2. użytkownik B odgaduje, czy x jest parzyste czy nieparzyste i przesyła swoje przypuszczenie do użytkownika A,
3. jeśli przypuszczenie użytkownika B jest poprawne, to wynikiem jest „reszka”, jeżeli nieprawdziwe, to „orzeł”, użytkownik A przesyła rezultat do użytkownika B,
4. użytkownik B potwierdza, że $y = f(x)$.

RZUT MONETĄ PRZY ZASTOSOWANIU LICZB CAŁKOWITYCH BLUMA

OPIS PROTOKOŁU:

1. użytkownik A generuje liczbę całkowitą Bluma n , losowe x względnie pierwsze z n , $x_0 = x^2 \pmod{n}$ i $x_1 = x_0^2 \pmod{n}$, przesyła n i x_1 do użytkownika B,
2. użytkownik B odgaduje, czy x_0 jest parzyste, czy nieparzyste,
3. użytkownik A przesyła do B x i x_0 ,
4. użytkownik B sprawdza, czy n jest liczbą całkowitą Bluma (użytkownik A powinien przekazać do B czynniki liczby n lub zrealizować protokół o wiedzy „zerowej” w celu przekonania go, że jest rzeczywiście liczbą całkowitą Bluma) i czy $x_0 = x^2 \pmod{n}$ oraz $x_1 = x_0^2 \pmod{n}$, jeżeli wszystko się zgadza, wynik rzutu zależy od poprawności przypuszczenia użytkownika B w kroku 2.

BEZPIECZEŃSTWO

W przypadku gdy n nie jest liczbą całkowitą Bluma, to użytkownik A może znaleźć takie x' , że

$$x'_0 = x'^2 \pmod{n} \quad \text{ i } \quad x_0'^2 \equiv x_0^2 \pmod{n}. \quad (16)$$

Jeżeli x_0 jest parzyste natomiast x'_0 nieparzyste (lub odwrotnie), to użytkownik A może oszukiwać.

14. Definicja, cechy i przykład protokołów (uczciwych) ślepych podpisów cyfrowych.

CECHY PODPISÓW

Podpis ręczny	Podpis cyfrowy	Ślepy podpis cyfrowy
Niepodrabialność	Niepodrabialność	Niepodrabialność
Autentyczność	Autentyczność	Autentyczność
Jest częścią integralną dokumentu, podpisu nie można ponownie użyć	Podpisu nie można ponownie użyć, może istnieć niezależnie	Podpisu nie można ponownie użyć, może istnieć niezależnie
Niezmienialność podpisanego dokumentu	Niezmienialność podpisanego dokumentu	Niezmienialność podpisanego dokumentu
Niezaprzeczalność	Niezaprzeczalność	Niezaprzeczalność
Podpisujący zna dokument	Podpisujący zna dokument	Podpisujący nie zna dokumentu
Jest taki sam dla wszystkich dokumentów	Jest różny dla różnych dokumentów	Jest różny dla różnych dokumentów
Występuje na ogół na ostatniej stronie dokumentu	Obejmuje cały dokument	Obejmuje cały dokument

RÓŻNICE MIĘDZY PODPISAMI CYFROWYMI

Tradycyjny podpis cyfrowy – podpisujący **wie** co podpisuje.

Ślepy podpis cyfrowy – podpisujący **nie wie** co zawiera podpisywany dokument.

CECHY ŚLEPYCH PODPISÓW CYFROWYCH

- ▶ poprawne zakończenie protokołu podpisu gwarantuje, że zawsze operacja weryfikacji zakończy się powodzeniem dla poprawnej pary wiadomość/podpis $(m, S(m))$;
- ▶ po zakończeniu protokołu nie ma możliwości powiązania pary $(m, S(m))$ z konkretnym wykonaniem protokołu (nie ma możliwości powiązania poprawnej pary $(m, S(m))$ z konkretną realizacją protokołu);
- ▶ jedyną możliwością dla atakującego wygenerowania poprawnej pary $(m, S(m))$ (bez znajomości klucza prywatnego podpisującego) jest wykonanie protokołu generacji podpisu z podpisującym jako drugą stroną.

WARUNKI WSTĘPNE:

użytkownik B jest osobą zaufaną – notariuszem świadczącym usługi potwierdzania faktu istnienia dokumentu; użytkownik A posiada dokument, do którego chce uzyskać podpis, jednak nie chce go ujawnić.

OPIS PROTOKOŁU:

1. użytkownik A pobiera dokument i wymnaża go przez liczbę losową, ta liczba nazywa się czynnikiem zaciemniającym;
2. użytkownik A przesyła zaciemniony dokument użytkownikowi B.
3. użytkownik B podpisuje zaciemniony dokument;
4. użytkownik A dzieli wynik przez czynnik zaciemniający, pozostawiając kopię oryginalnego dokumentu podpisanego przez użytkownika B.

WARUNKI DZIAŁANIA PROTOKOŁU:

funkcja podpisująca i funkcja mnożąca są przemienne

WŁASNOŚCI PROTOKOŁU:

- ▶ użytkownik B nie może uzyskać informacji co do zawartości dokumentu;
- ▶ użytkownik B nie może zaprzeczyć poprawności jego podpisu;
- ▶ użytkownik A ma pewne możliwości oszustwa gdy znajdzie inną liczbę która pomnożona przez inny dokument da ten sam wynik.

WADA

brak możliwości pewnej kontroli przez użytkownika B nad tym co podpisuje.

MODYFIKACJA

użytkownik B może otrzymać zestaw dokumentów zaciemnionych i po skontrolowaniu losowo wybranych może podpisać ostatni z nich.

OPIS PROTOKOŁU PO MODYFIKACJI:

1. użytkownik A przygotowuje np. dziesięć dokumentów do podpisu;
2. zaciemnia każdy z nich za pomocą odmiennych elementów zaciemniających;
3. użytkownik A przesyła zaciemnione dokumenty do użytkownika B.
4. użytkownik B wybiera losowo kilka dokumentów i żąda podania do nich elementów zaciemniających;
5. użytkownik A przesyła odpowiednie elementy zaciemniając;
6. użytkownik B usuwa zaciemnienie z wybranych dokumentów i sprawdza ich poprawność;
7. użytkownik B podpisuje wybrany przez siebie z puli nie skontrolowanej dokument i przesyła do użytkownika A.
8. użytkownik A usuwa zaciemnienie i uzyskuje podpisany dokument.

ZALETY W STOSUNKU DO POPRZEDNIEGO ROZWIĄZANIA:

1. użytkownik A musi przewidzieć co wybierze użytkownik B.

WYKORZYSTANIE RSA DO REALIZACJI ŚLEPYCH PODPISÓW CYFROWYCH

ZASADY OGÓLNE:

Użytkownik B posiada klucz jawny e , klucz prywatny d i moduł jawny n . Użytkownik A chce by użytkownik B podpisał na ślepo wiadomość m .

OPIS PROTOKOŁU:

1. użytkownik A wybiera losowo k z przedziału między 1 a n ;
2. zaciemnia m obliczając:

$$t \equiv m k^e \pmod{n} \quad (1)$$

wynik przesyła do użytkownika B.

3. użytkownik B podpisuje t :

$$t^d \equiv (m k^e)^d \pmod{n} \quad (2)$$

wynik przesyła do użytkownika A.

4. użytkownik A usuwa zaciemnienie t^d poprzez obliczenie:

$$s \equiv t^d / k \pmod{n} \equiv (m^d k) / k \pmod{n} \equiv m^d \pmod{n}. \quad (3)$$

ROZBUDOWANA WERSJA

Podpisujący posiada kilka kluczy jawnych

$$e_1, e_2, \dots, e_i \quad (4)$$

oraz odpowiadających im kluczy prywatnych

$$d_1, d_2, \dots, d_i. \quad (5)$$

Osoba oczekująca podpisania przez osobę podpisującą partycypuje w wyborze jakim kluczem posłuży się podpisujący, jednak ostatecznie podpisujący tego dokonuje.

OPIS PROTOKOŁU:

1. użytkownik A wybiera losowo r z przedziału między 1 a n ;

2. zaciemnia m obliczając:

$$t = m r^{e_1 e_2} \pmod{n} \quad (6)$$

wynik przesyła do użytkownika B.

3. użytkownik B podpisuje t stosując d_1 lub d_2 :

$$t^{d_1} = (m r^{e_1 e_2})^{d_1} \pmod{n} \quad (7)$$

albo

$$t^{d_2} = (m r^{e_1 e_2})^{d_2} \pmod{n} \quad (8)$$

wynik przesyła do użytkownika A.

4. użytkownik A usuwa zaciemnienie t^{d_1} lub t^{d_2} poprzez obliczenie:

$$m'_1 = (m r^{e_1 e_2})^{d_1} r^{-e_2} \pmod{n} \quad (9)$$

albo

$$m'_2 = (m r^{e_1 e_2})^{d_2} r^{-e_1} \pmod{n} \quad (10)$$

UCZCIWE ŚLEPE PODPISY CYFROWE

DEFINICJA

Uczciwe ślepe podpisy cyfrowe w stosunku do ślepych podpisów cyfrowych różnią się:

- ▶ istnieniem trzeciej zaufanej strony;
- ▶ zaufana trzecia strona ma możliwość usunięcia anonimowości – ma możliwość powiązania podpisu z wykonaniem protokołu generacji podpisu;

MODEL UCZCIWYCH ŚLEPYCH PODPISÓW CYFROWYCH

Na model uczciwych ślepych podpisów cyfrowych składają się:

1. protokół podpisu realizowany przez podpisującego i stronę ubiegającą się o podpis.
2. protokół usuwania anonimowości realizowany przez podpisującego i zaufaną trzecią stronę.

TYPY MODELI:

1. I – na podstawie zapisu realizacji protokołu podpisu zaufana trzecia strona dostarcza informacji pozwalających powiązać tą realizację z końcowym podpisem.
2. II – na podstawie pary (wiadomość, podpis) zaufana trzecia strona dostarcza informacji pozwalającej podpisującemu na identyfikację strony, dla której ten podpis został wygenerowany lub pozwala na powiązanie podpisy z realizacją protokołu jego generacji.

UCZCIWY PROTOKÓŁ PRZESYŁANIA NIEZAPRZECZALNEGO

Jest to modyfikacja protokołu przesyłania niezaprzeczalnego, w której zaufana trzecia strona ma możliwość określenia, którą wiadomość otrzymał odbierający.



ZAŁOŻENIA

Zaufana trzecia strona zna rozkład liczby złożonej n na czynniki pierwsze p i q ($n = pq$).

Liczba g jest resztą kwadratową w $\mathbb{Z}_n$ dużego rzędu.

Liczba h jest nieresztą kwadratową w $\mathbb{Z}_n^*$ dla której symbol Jacobiego jest dodatni.

Liczby n , g i h są znane publicznie.

OPIS PROTOKOŁU:

1. użytkownik A wybiera losowo $r \in \mathbb{Z}_n$, dla wyboru $c \in \{0, 1\}$ oblicza

$$t = g^r h^c \pmod{n} \quad (38)$$

i wysyła do B liczbę t .

2. B wybiera losowo $\alpha \in \mathbb{Z}_n$, oblicza

$$\begin{aligned} A &= g^\alpha \pmod{n}, \\ k_0 &= t^\alpha \pmod{n}, \\ k_1 &= (th^{-1})^\alpha \pmod{n}, \\ y_0 &= E_{k_0}(m_0), \\ y_1 &= E_{k_1}(m_1), \end{aligned} \quad (39)$$

i wysyła do A trójkę A, y_0, y_1 .

3. A oblicza

$$\begin{aligned} k_c &= A^r \pmod{n}, \\ m_c &= D_{k_c}(y_c). \end{aligned} \quad (40)$$

SCHEMAT UCZCIWEGO ŚLEPEGO PODPISU CYFROWEGO (MODYFIKACJA SCHEMATU FIATA SHAMIRA)

PARAMETRY SYSTEMU

Liczba $n = pq$, gdzie p, q są dwiema dużymi liczbami pierwszymi, spełniającą warunek $NWD(\phi(n), 3) = 1$.

Losowa liczba $y \in \mathbb{Z}_n^*$.

Funkcja skrótu H . Parametr bezpieczeństwa k (np. $k > 80$).

Dwa ciągi:

$$y_i = H(y + i) \pmod{n}, \quad x_i = y_i^{1/3} \pmod{n}, \quad i = 1, \dots, k \quad (41)$$

INICJACJA:

Para (n, y) jest kluczem publicznym podpisującego B. Strona B jako jedyna może obliczyć ciąg x_i , gdyż znając rozkład n potrafi wyznaczyć pierwiastki 3-go stopnia z liczb y_i w $\mathbb{Z}_n$.

1. Strona B wybiera losowo $r_1, r_2, \dots, r_k \in \mathbb{Z}_n^*$, oblicza

$$t = \prod_{i=1}^k r_i^3 \pmod{n} \quad (42)$$

i wysyła t do A.

OPIS PROTOKOŁU PODPISU:

1. Strona A wybiera losowo $a \in \mathbb{Z}_n^*$, oblicza

$$\begin{aligned}\hat{t} &= ta^3 \pmod{n}, \\ c &= H(\hat{t}||m).\end{aligned}\tag{43}$$

Niech c_i oznacza i -ty bit c .

2. A wykonuje z B uczciwy protokół przesyłania niezaprzeczalnego k razy, gdzie c_i ($i = 1, \dots, k$) są bitami wyboru A, B przygotowuje dwie wiadomości

$$\begin{aligned}m_0 &= r_i \pmod{n}, \\ m_1 &= r_i x_i \pmod{n}.\end{aligned}\tag{44}$$

A otrzymuje $s_i = m_{c_i}$.

3. A oblicza

$$\hat{s} = a \prod_{i=1}^k s_i \pmod{n}.\tag{45}$$

Podpisem pod m jest para $(\hat{s}, \hat{t})$.

WERYFIKACJA PODPISU:

1. W celu weryfikacji podpisu wystarczy sprawdzić, czy zachodzi równość:

$$\hat{s}^3 = \hat{t} \prod_{i=1}^n y_i^{c_i} \pmod{n}.\tag{46}$$

POWIĄZANIE PARY WIADOMOŚĆ–PODPIS Z WYKONANIEM PROTOKOŁU:

Jeżeli B przekaże zaufanej trzeciej stronie T zapis przebiegu protokołu podpisu, T wyznacza bity wyboru c_i i w konsekwencji c . Na tej podstawie każdy użytkownik znający c może zidentyfikować odpowiednią parę wiadomość–podpis.

15. Definicja, własności i przykład protokołów dowodów twierdzeń o wiedzy.

DEFINICJA

Służą między innymi do przedstawiania informacji o posiadaniu pewnych danych zarazem nie ujawniając ich osobie sprawdzającej, a także nikomu postronnemu.

Są jeszcze równoległe oraz niekonwersacyjne dowody

DEFINICJA:

Są to metody udowodnienia wiedzy pomiędzy użytkownikami ze zmniejszoną ilością wymiany informacji pomiędzy użytkownikami.

OPIS PROTOKOŁU:

1. użytkownik A wykorzystuje swoją informację i pewne liczby losowe do przekształcenia trudnego problemu w n odmiennych problemów izomorficznych, następnie wykorzystuje swoją informację i liczbę losową do rozwiązania tych n nowych trudnych problemów;
2. użytkownik A zobowiązuje się do rozwiązania n nowych trudnych problemów;
3. użytkownik A ujawnia użytkownikowi B n nowych trudnych problemów; użytkownik B nie może wykorzystać tego nowego problemu do uzyskania jakiegokolwiek informacji dotyczącej problemu pierwotnego lub jego rozwiązania;

DEFINICJA:

Stosuje się gdy trzeba przekonać stronę trzecią o swojej wiedzy nie prowadząc akurat z nią konwersacji.

OPIS PROTOKOŁU:

1. użytkownik A wykorzystuje swoją informację i pewne liczby losowe do przekształcenia trudnego problemu w n odmiennych problemów izomorficznych, następnie wykorzystuje swoją informację i liczbę losową do rozwiązania tych n nowych trudnych problemów;
2. użytkownik A zobowiązuje się do rozwiązania n nowych trudnych problemów;
3. użytkownik A wykorzystuje wszystkie zobowiązania jako wejścia do jednokierunkowej funkcji skrótu, następnie zachowuje on pierwsze n bitów wyniku działania tej funkcji jednokierunkowej;

WŁASNOŚCI DOWODÓW WIEDZY:

1. osoba dowodząca twierdzenia nie może oszukiwać sprawdzającego, jeżeli nie zna dowodu, to jej szanse na przekonanie sprawdzającego, że zna ona dowód, są znikome;
2. osoba sprawdzająca nie może oszukiwać osoby udowadniającej, nie uzyskuje ona najmniejszej wskazówki odnośnie do dowodu z wyjątkiem sytuacji, kiedy osoba udowadniająca zna dowód;
3. w szczególności osoba sprawdzająca nie może wykazywać poprawności dowodu komukolwiek innemu bez udowodnienia go samemu sobie na podstawie skrawków informacji;
4. osoba sprawdzająca nie dowiadyuje się od osoby dowodzącej niczego, czego nie mogłaby się dowiedzieć samodzielnie bez dowodzącej.

PROTOKÓŁ KONWERSACYJNY UDOWADNIANIA WIEDZY

Przykładowo użytkownik **A** zna część informacji i dodatkowo informacja ta jest rozwiązaniem trudnego problemu.

W TYM PRZYPADKU PROTOKÓŁ MA POSTAĆ:

1. użytkownik **A** używa swojej informacji i pewnej liczby losowej do przekształcenia trudnego problemu w inny trudny problem, który jest izomorficzny z funkcją pierwotną, następnie używa on swojej informacji i liczby losowej do rozwiązania nowego trudnego problemu;
2. użytkownik **A** dokonuje zobowiązania w stosunku do nowego rozwiązania, używając pewnego schematu zobowiązania bitowego;
3. użytkownik **A** ujawnia użytkownikowi **B** nowy problem trudny, użytkownik **B** nie może użyć tego nowego problemu do uzyskania jakiegokolwiek informacji dotyczącej problemu pierwotnego lub jego rozwiązania;

OPIS PROTOKOŁU CD.:

1. użytkownik **B** żąda od użytkownika **A** wykonania jednej z dwóch czynności:
 - 1.1 udowodnienia jemu, że stary i nowy problem są izomorficzne; albo
 - 1.2 otwarcia rozwiązania zawartego w zobowiązaniu w kroku 2) i udowodnienia, że jest to rozwiązanie nowego problemu;
2. użytkownik **A** wykonuje polecenie;
3. obydwaj użytkownicy **A** i **B** powtarzają powyższe kroki n-krotnie.